

# Introduction to tree algorithms | Graph Theory

---

**Channel:** WilliamFiset

**Video:** <https://youtu.be/1XC3p2zBK34?si=hJbLqJv3D8k8X3UV>

## Summary

This video introduces tree algorithms within graph theory, defining a tree as an undirected graph without cycles, characterized by  $N$  nodes and  $N-1$  edges. It explores various real-world applications, from computer file systems to web page structures and game theory, emphasizing their prevalence in data structures. The presentation then details different computational storage methods for trees, including edge lists, adjacency lists (preferred for efficiency), and adjacency matrices (generally discouraged due to space waste). The discussion progresses to rooted trees, which have a designated root and often directed edges, noting their ease of use for recursive algorithms. Further specializations like binary trees (at most two children) and binary search trees (maintaining a specific value invariant for efficient searching) are covered. Finally, it explains how to store rooted trees using parent/child pointers and a neat flattened array representation for binary trees, leveraging index-based calculations for parent and child access.

## Detailed Explanation

Hello student! Today, we're going to learn about trees, which are a fundamental concept in computer science and graph theory. Think of them as a special kind of graph.

First, what exactly *is* a tree? A tree is an undirected graph that has no cycles. Imagine a bunch of dots (nodes) connected by lines (edges); if you can start at a dot, follow lines, and return to your starting dot without repeating any line, that's a cycle. A tree doesn't have that! A key property of any tree is that if it has ' $N$ ' number of dots, it will always have exactly ' $N-1$ ' lines connecting them. This is an easy way to check if a graph is a tree.

Where do we see trees in the real world and in computer science? They're everywhere! Your computer's file system, with directories and subdirectories, is a tree. Social hierarchies, like a company's organizational chart from CEO down to employees, also form a tree. In programming, source code and math expressions are often broken down into 'Abstract Syntax Trees' for easy processing. Even every web page you visit is structured like a tree due to HTML's nested tags. Trees are also used in game theory to map out decisions, like in the famous Prisoner's Dilemma. Most importantly, in computer science, trees are fundamental data structures for organizing information

efficiently.

Now, how do we store these trees on a computer? First, we usually label all the nodes with numbers, typically starting from 0 up to  $N-1$ . One simple way is an 'edge list,' which is just a list of pairs of nodes that are connected. It's easy to store and go through, but it's not efficient if you want to quickly find all the neighbors of a single node.

A more popular and efficient way is the 'adjacency list.' Here, for each node, you store a list of all its direct neighbors. For example, if node 4 is connected to nodes 1, 5, and 8, then node 4's entry in the adjacency list would point to a list containing 1, 5, and 8. This is great for finding neighbors quickly.

Another option is an 'adjacency matrix,' which is a big grid ( $N$  by  $N$ ) where you mark a '1' in a cell if the row node and column node are connected. However, you should generally *avoid* using an adjacency matrix for trees. Why? Because trees have  $N-1$  edges, so most of the cells in the  $N$  by  $N$  matrix will be empty (zeros). It's a huge waste of memory, as you allocate  $N$ -squared memory but only use about  $2N$  cells.

Let's talk about 'rooted trees.' These are trees where one specific node is chosen as the 'root.' Think of it like the trunk of a tree, with everything else branching down from it. Most rooted trees have directed edges, meaning the connections point away from the root. Rooted trees are often easier to work with because their clear structure makes it simple to write recursive algorithms.

Related to rooted trees are 'binary trees.' In a binary tree, each node can have at most two 'child' nodes (nodes directly below it). You don't often see perfect binary trees naturally; computer scientists usually create them specifically for data structures because they guarantee efficient ways to add, remove, or access data.

Even more specialized are 'binary search trees' (BSTs). These are binary trees that follow a specific rule, called the 'binary search tree invariant.' This rule says that for any node 'X': all the values in its left subtree (children of its left child, and so on) must be less than or equal to 'X', and all the values in its right subtree must be greater than or equal to 'X'. This rule is incredibly powerful because it allows you to search for values very, very quickly. Sometimes, to avoid duplicate values, the rule is made stricter: left subtree values must be strictly less than 'X'.

Finally, how do we store rooted trees on a computer? You always need a reference (or pointer) to the root node so you can start traversing the tree. Then, each node typically stores a list of pointers to its 'children' nodes. Sometimes, it's also useful for a node to have a pointer to its 'parent' node, so you can move upwards in the tree. However, a parent pointer isn't always explicitly needed because recursive functions automatically manage returning to the parent's context on the call stack.

A very clever way to store a 'binary tree' is using a 'flattened array.' Imagine taking the tree and laying it out into a single list (an array). Each node is assigned an index in this array. The root node is always at index 0. The amazing thing about this format is that you can calculate the positions of a

node's children and parent directly from its own index! If a node is at index 'i', its left child will be at ' $2i + 1$ ' and its right child at ' $2i + 2$ '. Conversely, its parent will be at ' $(i - 1) / 2$ '. This allows for very efficient traversal without explicit pointers for children, though it can waste some space if the tree is very sparse or unbalanced.

## Key Learnings

- A tree is an undirected graph without cycles, characterized by N nodes and N-1 edges.
- Trees have widespread applications in computer science and the real world, from file systems to web pages.
- Adjacency lists are generally the most efficient and practical way to represent trees for neighbor queries.
- Adjacency matrices are space-inefficient for trees and should typically be avoided.
- Rooted trees simplify operations and enable easy recursive algorithm implementations due to their defined structure.
- Binary trees constrain nodes to at most two children, often used for efficient data structures.
- Binary Search Trees (BSTs) enforce an invariant that allows for very fast searching and data retrieval.
- Binary trees can be efficiently stored and traversed using a flattened array representation based on index calculations.

## Important Concepts

- Tree Definition (Undirected, No Cycles)
- Adjacency List Representation
- Rooted Trees
- Binary Trees
- Binary Search Tree Invariant
- Flattened Array Representation

## Applications

- Computer File Systems (directories/subdirectories)
- Social Hierarchies (organizational charts)
- Abstract Syntax Trees (for source code/math expressions)
- Web Page Rendering (HTML's nested tag structure)
- Game Theory (modeling decisions/courses of action)